The `flatMap` definition here maps over both `M` and `Option`, and flattens structures like `M[Option[M[Option[A]]]]` to just `M[Option[A]]`. But this particular implementation is specific to `Option`. And the general strategy of taking advantage of `Traverse` works only with traversable functors. To compose with `State` (which can't be traversed), for example, a specialized `StateT` monad transformer has to be written. There's no generic composition strategy that works for every monad.

See the chapter notes for more information about monad transformers.

## 12.8 Summary

In this chapter, we discovered two new useful abstractions, `Applicative` and `Traverse`, simply by playing with the signatures of our existing `Monad` interface. Applicative functors are a less expressive but more compositional generalization of monads. The functions `unit` and `map` allow us to lift values and functions, whereas `map2` and `apply` give us the power to lift functions of higher arities. Traversable functors are the result of generalizing the `sequence` and `traverse` functions we've seen many times. Together, `Applicative` and `Traverse` let us construct complex nested and parallel traversals out of simple elements that need only be written once. As you write more functional code, you'll learn to spot instances of these abstractions and how to make better use of them in your programs.

This is the final chapter in part 3, but there are many abstractions beyond `Monad`, `Applicative`, and `Traverse`, and you can apply the techniques we've developed here to discover new structures yourself. Functional programmers have of course been discovering and cataloguing for a while, and there is by now a whole zoo of such abstractions that captures various common patterns (*arrows*, *categories*, and *comonads*, just to name a few). Our hope is that these chapters have given you enough of an introduction to start exploring this wide world on your own. The material linked in the chapter notes is a good place to start.

In part 4 we'll complete the functional programming story. So far we've been writing libraries that might constitute the core of a practical application, but such applications will ultimately need to interface with the outside world. In part 4 we'll see that referential transparency can be made to apply even to programs that perform I/O operations or make use of mutable state. Even there, the principles and patterns we've learned so far allow us to write such programs in a compositional and reusable way.